

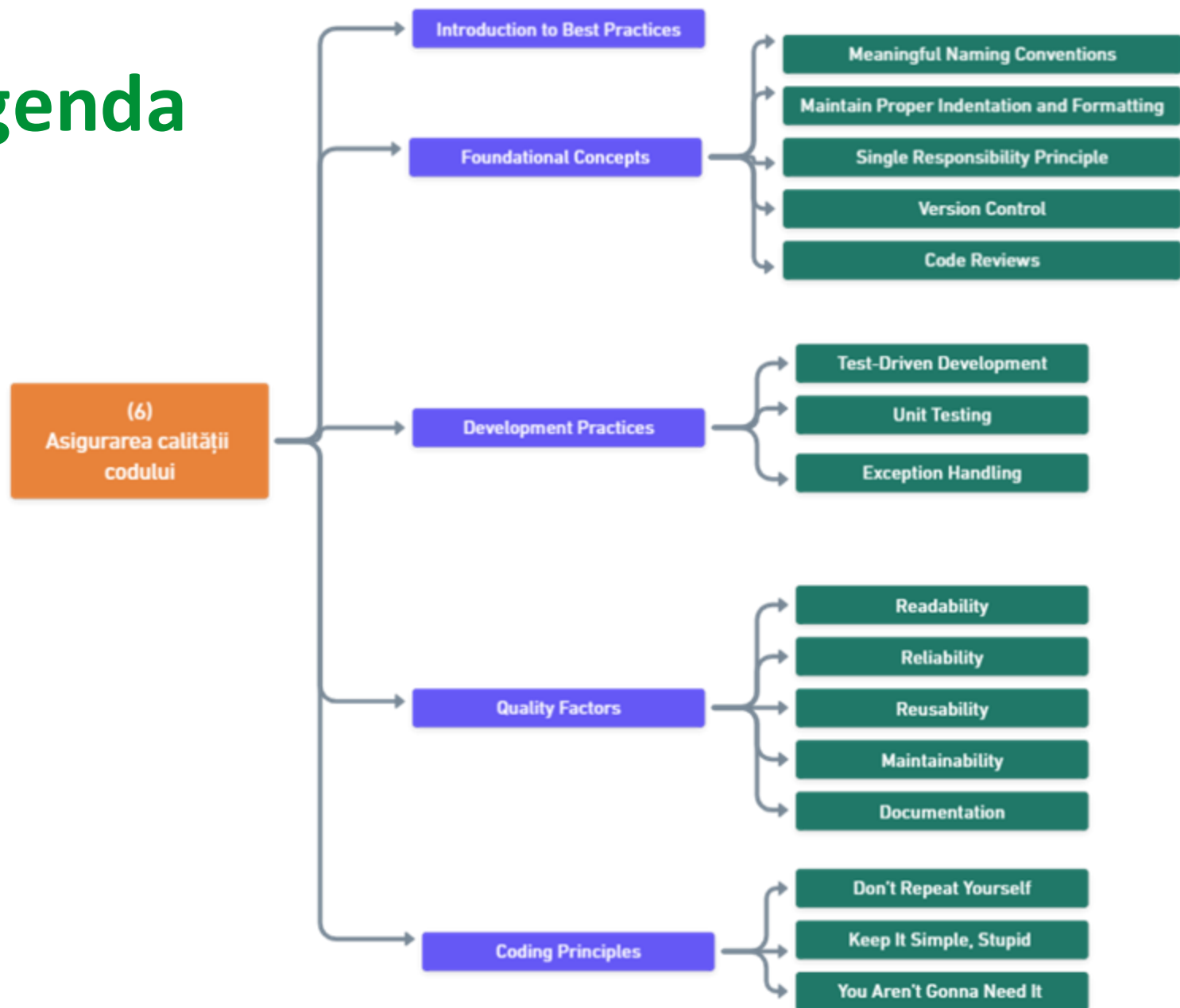
CODE QUALITY

Tehnici avansate de programare

Nicoleta Scrimint



Agenda



Introduction to Best Practices

- **Best practices** in C# software engineering encompass a set of **guidelines**, **principles** and **techniques** aimed at improving **code quality**, **maintainability**, and **scalability**.



 **The scope** of aiming to best practices is ensuring the **success** and **longevity** of software projects.

Meaningful Naming Conventions

- **Meaningful naming conventions** involve using **descriptive** and **self-explanatory** names for variables, methods, classes, and other code elements.

```
// Poorly named variable  
int x = 10;  
  
// Better named variable  
int numberOfStudents = 10;
```

- ① This practice enhances **code readability** and **reduces** the need for **comments** to explain the purpose of each element.

Maintain Proper Indentation and Formatting

- **Proper indentation and formatting** improve **code readability** by providing visual clues for code structure.

```
if (condition)
{
    DoSomething();
}
else
{
    DoSomethingElse();
}
```

 **Consistent indentation and formatting** style throughout the codebase make it easier for developers to **understand** and **maintain the code**.

Single Responsibility Principle

- The **Single Responsibility Principle** (SRP) states that a class or method should have **only one reason to change**, it should have a **single responsibility**.

```
// Good example
public class DatabaseManager
{
    public void Connect() { /* connect to database */ }
    public void Disconnect() { /* disconnect from database */ }
}

// Bad example: Mixing responsibilities
public class DatabaseManager
{
    public void Connect() { /* connect to database */ }
    public void AuthenticateUser() { /* authenticate user */ }
}
```

 This principle promotes **modular design**, improves **code maintainability**, and **reduces coupling** between different components of the system.

Version Control

- **Version control** systems like **Git** enable developers to **manage changes to source code** over time, **collaborate with team members**, and **maintain a history of changes**.



- Topics related to version control include **branching strategies**, **merging**, **resolving conflicts**, and using Git effectively in **team environments**.



Git is widely used for distributed version control due to its efficiency and powerful branching capabilities.

Code Reviews

- **Code reviews** involve examining code changes by peers to **identify bugs**, **improve code quality**, ensure **compliance to coding standards**, and **transfer knowledge** among team members.



 Code reviews are typically conducted before merging code changes into the main codebase.

Test-Driven Development

- **Test-Driven Development** (TDD) is a software development approach **where tests are written before writing the actual code**.
- The development process follows a cycle of **writing failing tests** (also **writing the minimum code to compile**), **writing the minimum code to pass** the tests, and **refactoring the code** while ensuring that all tests pass.



Test-Driven Development

- Write failing test cases:

```
public class CalculatorTests
{
    [Test]
    public void Add_TwoNumbers_ReturnsCorrectSum()
    {
        Calculator calculator = new Calculator();
        int result = calculator.Add(3, 5);
        Assert.AreEqual(8, result);
    }

    [Test]
    public void Subtract_TwoNumbers_ReturnsCorrectDifference()
    {
        Calculator calculator = new Calculator();
        int result = calculator.Subtract(8, 3);
        Assert.AreEqual(5, result);
    }
}
```

Test-Driven Development

- Implement the main class (just to make tests compile):

```
public class Calculator
{
    public int Add(int a, int b)
    {
        // Returning incorrect result intentionally
        return a
    }

    public int Subtract(int a, int b)
    {
        // Returning incorrect result intentionally
        return a
    }
}
```

Test-Driven Development

- Run the tests (they should fail initially).
- Implement just enough code to make the tests pass:

```
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public int Subtract(int a, int b)
    {
        return a - b;
    }
}
```

- Refactor code if necessary

Unit Testing

- **Unit testing** is a software testing technique where **individual units** or **components** of a software application are **tested in isolation** to ensure they function correctly.
- This practice helps **detect defects early** in the development cycle, **facilitates code refactoring**, and **improves code quality**.



Unit Testing

Topics related to unit testing include:

- AAA pattern
- Mocking
- Stubbing
- Code Coverage
- Best Practices



Unit Testing

AAA Pattern (Arrange-Act-Assert)

- This pattern structures unit tests into three main phases: **Arrange**, **Act**, and **Assert**.
 - In the **Arrange** phase, you set up the test environment and initialize any necessary objects.
 - The **Act** phase involves invoking the method or behavior being tested.
 - Finally, in the **Assert** phase, you verify the expected behavior or outcome.

Unit Testing

```
[Test]
public void Add_TwoNumbers_ReturnsCorrectSum()
{
    // Arrange
    Calculator calculator = new Calculator();
    int a = 3;
    int b = 5;

    // Act
    int result = calculator.Add(a, b);

    // Assert
    Assert.AreEqual(8, result);
}
```


Unit Testing

```
[Test]
public void Subtract_TwoNumbers_ReturnsCorrectDifference()
{
    // Arrange
    Calculator calculator = new Calculator();
    int a = 8;
    int b = 3;

    // Act
    int result = calculator.Subtract(a, b);

    // Assert
    Assert.AreEqual(5, result);
}
```

Unit Testing

Unit Testing Mocking

- **Mocking** is the technique of creating simulated objects that mimic the behavior of real dependencies, enabling isolation and controlled testing of units within software systems.

```
// Interface representing a data repository
public interface IRepository
{
    void SaveData(string data);
}
```

Unit Testing

```
// Class under test that depends on IRepository
public class DataManager
{
    private readonly IRepository _repository;

    public DataManager(IRepository repository)
    {
        _repository = repository;
    }

    public void ProcessAndSaveData(string data)
    {
        // Process data
        // ...

        // Save processed data using IRepository
        _repository.SaveData(data);
    }
}
```

Unit Testing

```
[TestFixture]
public class DataManagerTests
{
    [Test]
    public void ProcessAndSaveData_WhenCalled_CallsSaveDataOnDataRepository()
    {
        // Arrange
        var mockRepository = new Mock<IDataRepository>();
        var dataManager = new DataManager(mockRepository.Object);
        string testData = "Test data";

        // Act
        dataManager.ProcessAndSaveData(testData);

        // Assert
        mockRepository.Verify(x => x.SaveData(testData), Times.Once);
    }
}
```

Unit Testing

Unit Testing Stubbing

- **Unit Testing Stubbing** is a technique used in unit testing to replace real dependencies with simplified implementations, known as stubs, for the purpose of testing. Stubs are minimalistic versions of the real dependencies that provide predetermined responses to method calls.

```
// Interface representing a payment gateway
public interface IPaymentGateway
{
    bool ProcessPayment(decimal amount);
}
```

Unit Testing

```
// Class under test that depends on IPaymentGateway
public class PaymentProcessor
{
    private readonly IPaymentGateway _paymentGateway;

    public PaymentProcessor(IPaymentGateway paymentGateway)
    {
        _paymentGateway = paymentGateway;
    }

    public bool ProcessPayment(decimal amount)
    {
        // Call the payment gateway to process payment
        return _paymentGateway.ProcessPayment(amount);
    }
}
```



Unit Testing

```
// Stub implementation of IPaymentGateway for unit testing
public class PaymentGatewayStub : IPaymentGateway
{
    public bool ProcessPayment(decimal amount)
    {
        // Stub implementation always returns true (successful payment)
        return true;
    }
}
```

Unit Testing

```
[TestFixture]
public class PaymentProcessorTests
{
    [Test]
    public void ProcessPayment_WithStubbedPaymentGateway_ReturnsTrue()
    {
        // Arrange
        var paymentGatewayStub = new PaymentGatewayStub();
        var paymentProcessor = new PaymentProcessor(paymentGatewayStub);
        decimal amount = 100;

        // Act
        bool result = paymentProcessor.ProcessPayment(amount);

        // Assert
        Assert.IsTrue(result);
    }
}
```


Unit Testing

Unit Testing Coverage

- **Unit Testing Coverage** measures the proportion of code executed by unit tests, indicating how completely the tests exercise the software's functionality.

① It helps identify untested areas, guiding test improvement efforts and providing insight into code quality and reliability.



Unit Testing

Statement Coverage

```
public void ValidateUser(string username, string password)
{
    if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
    {
        throw new ArgumentException("Username and password are required.");
    }
    // Validate user credentials...
}
```

```
[Test]
public void ValidateUser_NullUsername_ThrowsArgumentException()
{
    // Arrange
    UserManager userManager = new UserManager();

    // Act & Assert
    Assert.Throws<ArgumentException>(() => userManager.ValidateUser(null, "password")
}
```



Unit Testing

Line Coverage

```
public int Add(int a, int b)
{
    if (a > 0)
    {
        return a + b;
    }
    else
    {
        return b;
    }
}
```

```
[Test]
public void Add_PositiveInput_ReturnsSum()
{
    // Arrange
    Calculator calculator = new Calculator();

    // Act
    int result = calculator.Add(3, 5);

    // Assert
    Assert.AreEqual(8, result);
}
```

Unit Testing

Branch Coverage

```
public string GetGrade(int score)
{
    if (score >= 90)
    {
        return "A";
    }
    else if (score >= 80)
    {
        return "B";
    }
    else
    {
        return "C";
    }
}
```

```
[Test]
public void GetGrade_ReturnsGradeBasedOnScore()
{
    // Arrange
    GradingSystem gradingSystem = new GradingSystem();

    // Act
    string gradeA = gradingSystem.GetGrade(95);
    string gradeB = gradingSystem.GetGrade(85);
    string gradeC = gradingSystem.GetGrade(75);

    // Assert
    Assert.AreEqual("A", gradeA);
    Assert.AreEqual("B", gradeB);
    Assert.AreEqual("C", gradeC);
}
```



Unit Testing

Condition Coverage

```
public string GetDiscountCategory(int age, bool isStudent)
{
    if (age >= 65 || isStudent)
    {
        return "Senior/Student Discount";
    }
    else
    {
        return "Regular Price";
    }
}
```

```
[Test]
public void GetDiscountCategory_Senior_ReturnsDiscountCategory()
{
    // Arrange
    DiscountCalculator discountCalculator = new DiscountCalculator();

    // Act
    string result = discountCalculator.GetDiscountCategory(70, false);

    // Assert
    Assert.AreEqual("Senior/Student Discount", result);
}
```



Unit Testing

Unit Testing Good Practices



- Test **Readability**:
 - Write clear, descriptive test names and use meaningful assertions to communicate the purpose and expected behavior of each test.
- Test **Isolation**:
 - Ensure that each test is independent and isolated from external dependencies, such as databases, file systems, or network services.
- Test **Coverage**:
 - Aim for high test coverage to verify critical paths, edge cases, and error-handling scenarios.



Unit Testing



- Test **Automation**:
 - Automate the execution of unit tests to enable frequent and consistent testing, especially as part of continuous integration (CI) and continuous delivery (CD) pipelines.



- Test **Maintainability**:
 - Keep tests simple, focused, and maintainable by avoiding overly complex setups, redundant assertions, or unnecessary dependencies.

Attendance – Course "Advanced
programming techniques" 26
March 2026 – Course 6



Observable behavior to be tested

In process

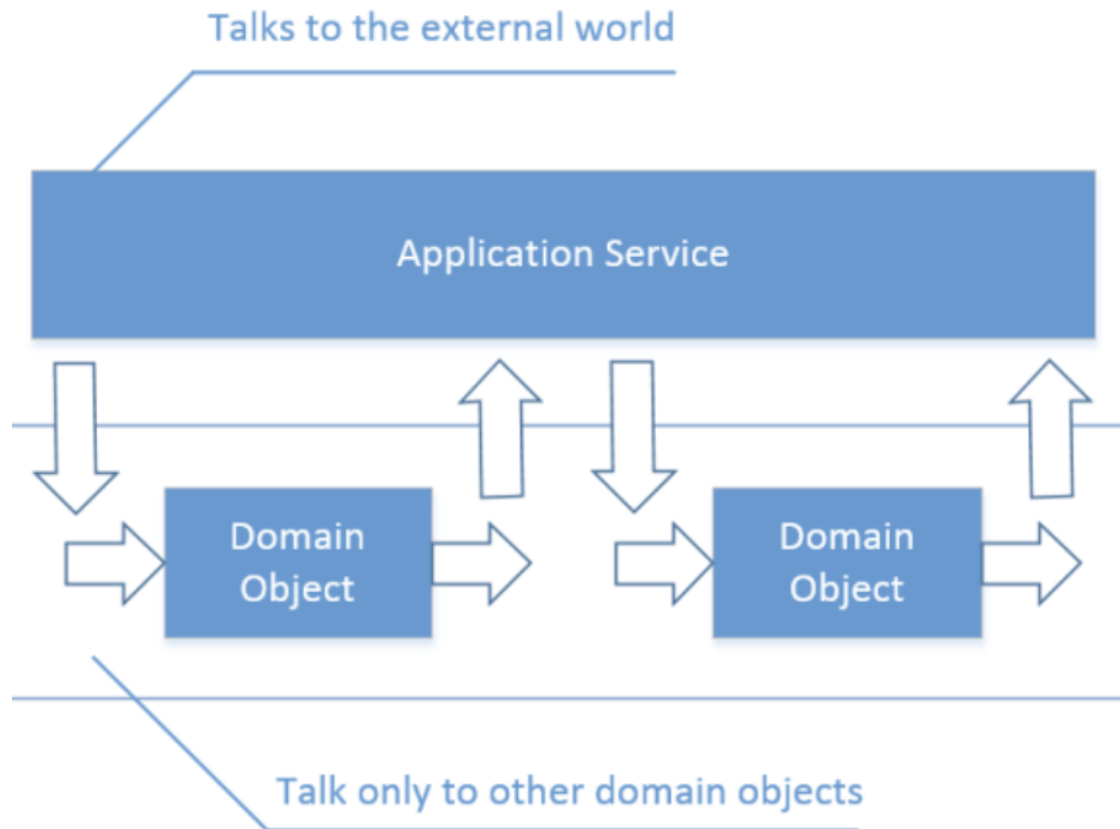
- State: Domain model state changes, e.g. order status after a checkout
- Result: Computation outcome from domain logic, e.g. the total of an order, after applied discount



Out of process

- Result: Retrieving data from the database
- State: Saving data in the database
- Interaction: Calling third party systems, e.g. pay the order with a payment provider

Integration tests



After isolating domain model from volatile dependencies

• <https://enterprisecraftsmanship.com/posts/pragmatic-integration-testing/>

Integration tests

- Covers coordination logic which resides in application services
- Dependencies you control, e.g. database
 - Use real databases in order to test
 - Database transactions
 - Primary key constraints
 - Unique indexes
- Dependencies you don't control
 - Use stubs for retrieving data from external systems
 - Use mocks to assert interaction with external systems



Test frameworks

- Write and run tests
 - MsTest
 - XUnit
- Test assertions
 - Shouldly
- Mocks and stubs
 - FakeItEasy



Test frameworks - XUnit

①

```
namespace MyFirstUnitTests;

public class UnitTest1
{
    [Fact]
    public void Test1()
    {
        Assert.True(true);
    }
}
```

```
[Theory]
[InlineData(3)]
[InlineData(5)]
[InlineData(6)]
public void MyFirstTheory(int value)
{
    Assert.True(IsOdd(value));
}

bool IsOdd(int value)
{
    return value % 2 == 1;
}
```

Test frameworks - Shouldly



Shouldly Logo

CI passing [↗](#) downloads 99M [↗](#) nuget v4.3.0 [↗](#) [gitter](#) [join chat](#) [↗](#)

Shouldly is an assertion framework which focuses on giving great error messages when the assertion fails while being simple and terse.

This is the old *Assert* way:

`Assert.That(contestant.Points, Is.EqualTo(1337));`

For your troubles, you get this message, when it fails:

Expected 1337 but was 0

How it **Should** be:

`contestant.Points.ShouldBe(1337);`



Test frameworks - FakeItEasy

```
// Creating a fake object is just dead easy!  
// No mocks, no stubs, everything's a fake!  
var lollipop = A.Fake<ICandy>();  
var shop = A.Fake<ICandyShop>();  
// Easily set up a call to return a value  
A.CallTo(() => shop.GetTopSellingCandy())Returns(lollipop);  
// Use your fake as you would an instance of the faked type.  
① var developer = new SweetTooth();  
developer.BuyTastiestCandy(shop);  
// Asserting uses the same syntax as configuring calls.  
// There's no need to learn another syntax.  
A.CallTo(() => shop.BuyCandy(lollipop))MustHaveHappened();
```

Exception Handling

- **Exception handling** involves managing **unexpected** or **exceptional conditions** that occur during the execution of a program.

```
try
{
    // Code that may throw an exception
}
catch (Exception ex)
{
    logger.LogError(ex, "An error occurred");
    throw; // Re-throw the exception
}
```



Proper exception handling helps maintain application **stability**, **prevents crashes**, and provides **meaningful error** messages to users or developers for debugging purposes.

Exceptions for exceptional cases

- Errors we don't know how to deal with
- Throw an exception to state an unexpected situation
- Global exception handler
 - Catch unexpected exceptions from own source code or from external code: libraries, third party system integrations
 - Log the exception
 - Discard the current operation completely and return to the caller



Results for errors known to be handled

- When you know how to deal with exceptions, catch them at the lowest level possible.
- <https://enterprisecraftsmanship.com/posts/functional-c-handling-failures-input-errors/>



Results for errors known to be handled

```
public class Result
{
    public bool Success { get; private set; }
    public string Error { get; private set; }
    public bool Failure { /* ... */ }

    protected Result(bool success, string error) { /* ... */ }

    public static ResultFail(string message) { /* ... */ }

    public static Result<T> Ok<T>(T value) { /* ... */ }
}

public class Result<T> : Result
{
    public T Value { get; set; }

    protected internal Result(T value, bool success, string error)
        : base(success, error)
    {
        /* ... */
    }
}
```



Quality Factors

- **Quality factors** contains various aspects of software quality, including **readability**, **reliability**, **reusability**, **maintainability**, and **documentation**.



These factors collectively contribute to the overall quality of the software product and its ability to meet user requirements and expectations.

Readability

Readability refers to how **easy** it is to **understand the code**.



Key Points:

- Use meaningful names for variables, methods, and classes.
- Keep code well-organized with proper indentation and formatting.
- Follow consistent coding style and naming conventions.

Reliability

Reliability refers to the ability of the software to perform **consistently** and **predictably** under various conditions.



Key Points:

- Implement robust error handling and exception management.
- Perform thorough testing, including unit tests, integration tests, and system tests.
- Handle edge cases and boundary conditions.
- Ensure compatibility with different environments and configurations.

Reusability

Reusability refers to the **ability** to **reuse code, components** in different parts of the software or in other projects.



Key Points:

- Identify common functionalities and extract them into reusable components or libraries.
- Design classes and methods with clear interfaces and separation of concerns.
- Use inheritance, interfaces, and design patterns to facilitate code reuse.
- Document and maintain reusable components for easy discovery and usage.

Reusability

- It is recommended to not create reusable libraries for business concerns, e.g. checkout an order
- It is recommended to create reusable libraries for infrastructure concerns, e.g. database persistence, logging, configuration

Maintainability

Maintainability refers to how **easy** it is to **modify**, **extend**, and **update** the software over time.



Key Points:

- Follow coding standards and best practices to ensure consistency and readability.
- Modularize code into smaller, cohesive components with well-defined responsibilities.
- Minimize dependencies and coupling between different modules or layers.
- Document design decisions, architectural patterns, and implementation details.

Documentation

Documentation refers to the **supplementary materials** that describe the software's **functionality**, **architecture**, **design**, and **usage**.



Key Points:

- Create user manuals and release notes to assist customers
- Create developer setup guides, how-to articles to support developers
- Maintain up-to-date documentation alongside code changes.
- Document technical decisions for chosen libraries, integration with third party systems

Don't Repeat Yourself

Don't Repeat Yourself (DRY) Principle

- DRY emphasizes the importance of **avoiding code duplication** by extracting **common functionality** into reusable components.
- DRY promotes code **maintainability**, **readability**, and **reduces** the **risk** of inconsistencies and bugs.

Don't Repeat Yourself

```
// Bad Example (Repetitive Code)
public class UserManager
{
    public void ValidateUsername(string username)
    {
        if (string.IsNullOrEmpty(username))
        {
            throw new ArgumentException("Username cannot be empty.");
        }
    }

    public void ValidatePassword(string password)
    {
        if (string.IsNullOrEmpty(password))
        {
            throw new ArgumentException("Password cannot be empty.");
        }
    }
}
```



Don't Repeat Yourself

```
// Good Example (Using DRY Principle)
public class UserManager
{
    public void ValidateInput(string input, string fieldName)
    {
        if (string.IsNullOrEmpty(input))
        {
            throw new ArgumentException(fieldName + " cannot be empty.");
        }
    }

    public void ValidateUsername(string username)
    {
        ValidateInput(username, "Username");
    }

    public void ValidatePassword(string password)
    {
        ValidateInput(password, "Password");
    }
}
```



Keep It Simple, Stupid

Keep It Simple, Stupid (KISS) Principle

- KISS advocates for **simplicity** in design and implementation. It suggests that simplicity leads to **better understanding**, **maintenance**, and **reduced risk** of errors.
- KISS encourages **avoiding unnecessary complexity** and keeping **solutions** as **straightforward** as possible.

Keep It Simple, Stupid

```
// Bad Example (Overly Complex Code)
public bool IsLeapYear(int year)
{
    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
    {
        return true;
    }
    else
    {
        return false;
    }
}
```

Keep It Simple, Stupid

```
// Good Example (Simplified Code)
public bool IsLeapYear(int year)
{
    return DateTime.IsLeapYear(year);
}
```


You Aren't Gonna Need It

You Aren't Gonna Need It (YAGNI) Principle

- YAGNI advises **against adding features**, functionality, or complexity **until it's necessary**. It encourages developers to focus on current requirements and **avoid speculative implementation**.
- YAGNI promotes **agility**, **simplicity**, and **avoids unnecessary code** and maintenance overhead.

You Aren't Gonna Need It

```
// Bad Example (Speculative Implementation)
public class ShoppingCart
{
    public void AddItem(Item item)
    {
        // Implementation for adding items
    }

    public void RemoveItem(Item item)
    {
        // Implementation for removing items
    }

    public void UpdateItemQuantity(Item item, int quantity)
    {
        // Implementation for updating item quantity
    }

    // More methods...
}
```

You Aren't Gonna Need It

```
// Good Example (Keeping It Simple)
public class ShoppingCart
{
    private List<Item> items = new List<Item>();

    public void AddItem(Item item)
    {
        items.Add(item);
    }

    // No need for remove or update methods until they are actually required
}
```



3 things that will make or break your project

- <https://enterprisecraftsmanship.com/posts/3-things-that-will-make-or-break-your-project/>